

Javaプログラミング入門

Introduction to Java Programming

第10回 継承・オーバーライド

Class 10. Inheritance and Overriding

早稲田大学 基幹理工学部 情報通信学科 講師

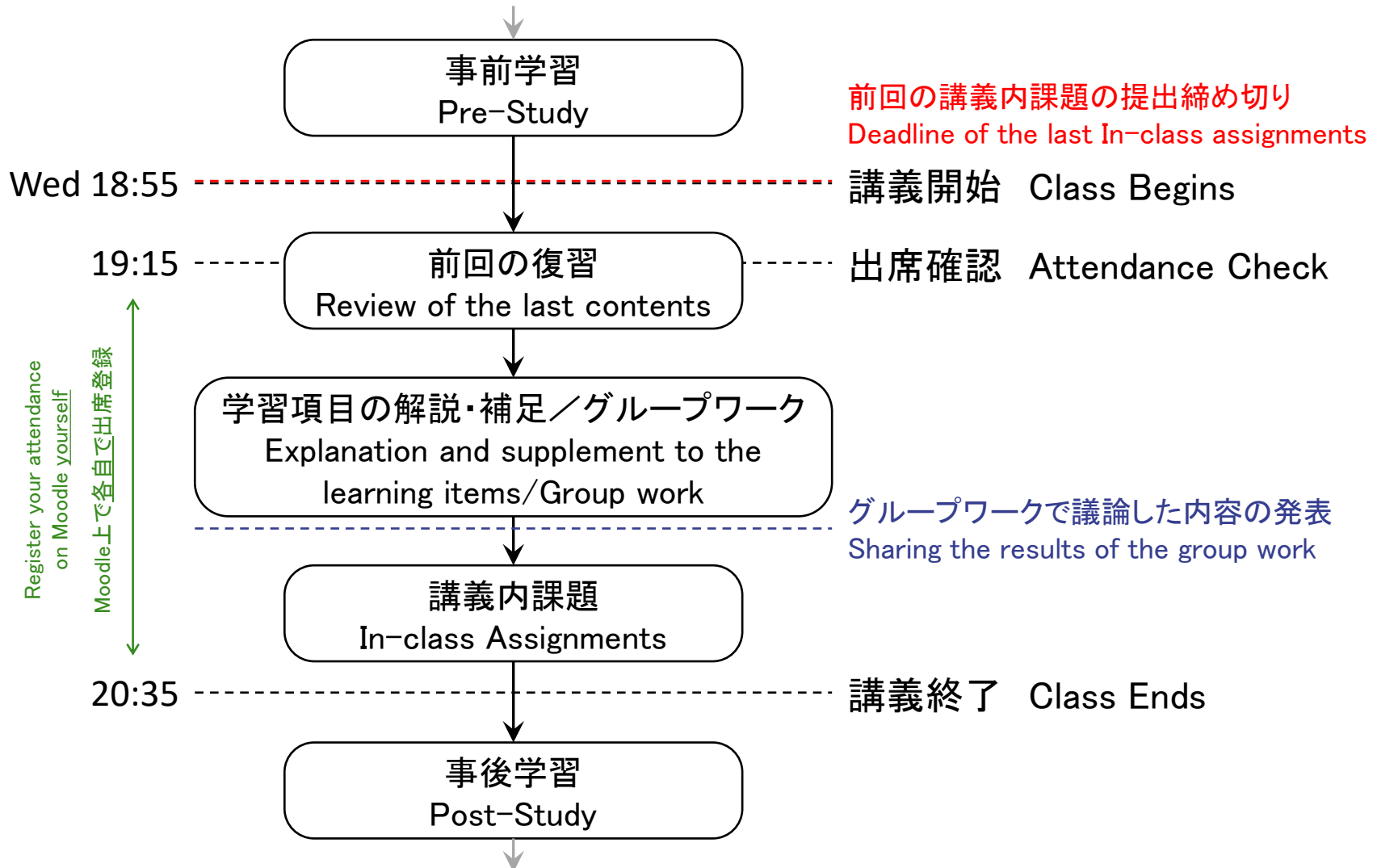
吉井 一駿

Kazutoshi YOSHII

講義の流れ Flow of the Class

本講義は反転講義です。Moodle上にある予習ビデオを視聴の上講義に臨むこと。

This course is a flipped classroom. Watch the video before each class.



第10回講義 Class 10

継承・オーバーライド Inheritance and Overriding



第9回 継承・オーバーライド 00:05:46



Part9 Inheritance and Overriding 00:04:54

到達目標：下記キーワードの理解

Goals for the Next Class: Understanding the Following Topics

継承 Inheritance スーパークラス Superclass サブクラス
Subclass オーバーライド Override `super.method()` `super()`



前回の復習

Review of the previous class

講義内課題 9A In class Assignment 9A

- 右の3ファイルを作成して下さい。
Please create the following three files.
 - 1. MonsterBattleGame_WithClass.java
 - 2. GameCharacter.java
 - 3. BattleProcess.java
- HPが0未満にならないように、メソッドreceiveDamageを修正して下さい。
Modify the receiveDamage method to ensure that HP never becomes less than 0.

下で指定されたコマンドでコンパイル&実行できるように、ファイルを修正して下さい。
コンパイル&実行できる状態で提出して下さい。

Modify the file so that it can be compiled and run using the following commands.
Please submit the program that can be compiled and run.

Commands

```
$ javac Assignment9A_[Student ID].java GameCharacter.java BattleProcess.java  
$ java Assignment9A_[Student ID]
```

提出ファイル名 Submission file name: Assignment9A_[Student ID].java, GameCharacter.java BattleProcess.java

提出方法 Submission method: Moodle

提出期限 Submission deadline: 第10回講義の開始まで. By the start of the class 10.

講義内課題 9A In class Assignment 9A

- 右の3ファイルを作成して下さい。
Please create the following three files.

1. MonsterBattleGame_WithClass.java
2. GameCharacter.java
3. BattleProcess.java

- HPが0未満にならないように、メソッドreceiveDamageを修正して下さい。
Modify the receiveDamage method to ensure that HP never becomes less than 0.

MonsterBattleGame_WithClass.java

```
import java.util.Random; // Randomクラスの使用 / Use the Random class
import java.util.Scanner; // Scannerクラスの使用 / Use the Scanner class

public class MonsterBattleGame_WithClass {

    public static void main(String args[]){ // プログラムの実行開始地点 / The starting point of program execution
        // -----
        // プログラム実行の準備 / Program execution setup
        Scanner scan = new Scanner(System.in); // Enter入力を受け取るための準備 / Prepare for reading Enter input
        Random rand = new Random(100); // 乱数生成処理の初期化 / Initialize random number generation

        // -----
        // インスタンスの生成と参照 / Creating and referring to instances
        GameCharacter player = new GameCharacter("Player", 100, 30); // GameCharacterクラスの実体(インスタンス)を生成し、変数playerから参照 / Create an instance of the GameCharacter class and refer to it from the variable player
        GameCharacter monster = new GameCharacter("Monster", 100, 30); // GameCharacterクラスの実体(インスタンス)を生成し、変数monsterから参照 / Create an instance of the GameCharacter class and refer to it from the variable monster

        // ゲームタイトルの表示 / Display the game title
        System.out.println("~~~~~ MonsterBattleGame_WithClass ~~~~");
        System.out.println("~~~~~");
        System.out.println("~~~~~");
        player.print(); // プレイヤー情報の表示 / Display the player info
        monster.print(); // モンスター情報の表示 / Display the monster info

        // -----
        // バトル進行 / Battle control
        System.out.println("# バトル開始 / Battle start");
        while (true) {

            System.out.println("# Enterを入力して攻撃 / Press Enter to attack");
            scan.nextLine(); // プレイヤーの入力待ち / Wait for player input

            // -----
            // プレイヤーがモンスターを攻撃 / The player attacks the monster
            BattleProcess.attack(player, monster, rand);

            // 戦闘不能判定 / Check whether the character is defeated
            if (BattleProcess.isDefeated(monster)) { // モンスターが戦闘不能の場合 / If monster is defeated
                BattleProcess.printResult(player, monster); // プレイヤーの勝利 / Player win
                break; // ループを抜ける / Exit the loop
            }

            // -----
            // モンスターがプレイヤーを攻撃 / The monster attacks the player
            BattleProcess.attack(monster, player, rand);

            // 戦闘不能判定 / Check whether the character is defeated
            if (BattleProcess.isDefeated(player)) { // プレイヤーが戦闘不能の場合 / If player is defeated
                BattleProcess.printResult(monster, player); // モンスターの勝利 / Monster win
                break; // ループを抜ける / Exit the loop
            }
        }

        // -----
        // 終了処理 / Exit handling
        scan.close(); // Scannerを閉じる / Close the Scanner
    }
}
```

講義内課題 9A In class Assignment 9A

- 右の3ファイルを作成して下さい。
Please create the following three files.

1. MonsterBattleGame_WithClass.java
2. GameCharacter.java
3. BattleProcess.java

- HPが0未満にならないように、メソッドreceiveDamageを修正して下さい。
Modify the receiveDamage method to ensure that HP never becomes less than 0.

GameCharacter.java

```
// =====  
//  
// ゲームキャラクタークラス / Game character class  
//  
// =====  
public class GameCharacter {  
  
    // -----  
    // フィールド / Fields  
    // - インスタンスが持つデータ / Fields are data stored in each instance  
    String name;        // キャラクター名 / Character name  
    int hp;              // キャラクターのHP / Character HP  
    int attack_power;    // キャラクターの攻撃力 / Character Attack power  
  
    // -----  
    // コンストラクタ / Constructor  
    // - コンストラクタの名前はクラス名と同じにする / The constructor name must be the same as the class name  
    // - コンストラクタはインスタンスを生成するときに最初に実行される / A constructor runs when an instance is created  
    public GameCharacter(String character_name, int character_hp, int character_attack_power) {  
        name = character_name;        // 引数character_nameをフィールドnameに代入 / Assign the argument character_name to the field name  
        hp = character_hp;            // 引数character_hpをフィールドhpに代入 / Assign the argument character_hp to the field hp  
        attack_power = character_attack_power; // 引数character_attack_powerをフィールドattack_powerに代入 / Assign the argument character_attack_power to the field attack_power  
    }  
  
    // -----  
    // パラメータを表示するメソッド / Method for printing parameters  
    public void print() {  
        System.out.println("[ " + name + " ]");  
        System.out.println("- HP: " + hp);  
        System.out.println("- Attack Power: " + attack_power);  
        System.out.println("-----");  
    }  
  
    // -----  
    // ダメージを受けるメソッド / Method for receiving damage  
    public void receiveDamage(int damage) {  
        hp -= damage;        // 受けたダメージ分だけHPを減らす / Reduce HP by the damage value  
        if (hp < 0) { hp = 0; } // HPは0以上 / HP is larger than 0  
    }  
}
```

講義内課題 9A In class Assignment 9A

- 右の3ファイルを作成して下さい。
Please create the following three files.

1. MonsterBattleGame_WithClass.java
2. GameCharacter.java
3. BattleProcess.java

- HPが0未満にならないように、メソッドreceiveDamageを修正して下さい。
Modify the receiveDamage method to ensure that HP never becomes less than 0.

BattleProcess.java

```
import java.util.Random; // Randomクラスの使用 / Use the Random class

// =====
// 戦闘プロセスクラス / Battle process class
//
// - BattleProcessはHPなどのフィールドを持たないため、インスタンスを作らずに使う / Since BattleProcess has no fields such as HP, these methods are used without creating an instance
// - staticを付けると、BattleProcessインスタンスを作らずにメソッドを呼び出せる / By using static, this method can be called without creating a BattleProcess instance
// =====
public class BattleProcess {

    // -----
    // 攻撃処理メソッド / Attack process method
    // - attackerがtargetを攻撃 / The attacker attacks the target
    public static void attack(GameCharacter attacker, GameCharacter target, Random rand) {

        // -----
        // 攻撃者を表示 / Show the attacker
        System.out.println(attacker.name + "の攻撃 / Attack by " + attacker.name);

        // -----
        // 攻撃ダメージの計算 / Calculate attack damage
        int attack_damage = rand.nextInt(attacker.attack_power);

        // -----
        // 攻撃対象にダメージを与える / Deal damage to the target
        target.receiveDamage(attack_damage);

        // -----
        // 攻撃結果を表示 / Show attack result
        System.out.println("> " + target.name + "に" + attack_damage + "のダメージ! / Dealt " + attack_damage + " damage to the " + target.name + "!");
        System.out.println("> " + target.name + "の残りHP: " + target.hp + " / " + target.name + " HP: " + target.hp);
    }

    // 戦闘不能を判定するメソッド / Method for checking whether the character is defeated
    public static boolean isDefeated(GameCharacter character) {
        return character.hp <= 0; // HPが0以下ならtrueを返す / Return true if HP is 0 or less
    }

    // 戦闘結果を表示するメソッド / Method for displaying the battle result
    public static void printResult(GameCharacter winner, GameCharacter loser) {
        System.out.println(" ");
        System.out.println("-----");
        System.out.println("結果 / Battle Result ");
        System.out.println("-----");
        System.out.println("> 勝利 / Win : " + winner.name);
        System.out.println("> 敗北 / Lose: " + loser.name);
    }
}
```


講義内課題 9B In class Assignment 9B

3体の敵モンスターと戦うゲーム, `MonsterBattleGame_WithClass_2.java` を作成しなさい。

乱数のシードは999, Classを必ず使用してかつ以下の条件を満たすこと。

- プレイヤーのHPは300, 攻撃力は60. モンスターのHPは100, 攻撃力は20.
- プレイヤーは20%, モンスターは10%の確率で攻撃が失敗するようにプログラムすること.
- 攻撃順: プレイヤーが各モンスターを攻撃 → 各モンスターがプレイヤーを攻撃 → ...

提出の際は, 下で指定されたコマンドでコンパイル&実行できる状態で提出して下さい。

Create `MonsterBattleGame_WithClass_2.java`, a game in which the player fights against three enemy monsters.

Set the random seed to 999, and the program must be implemented using class and the following conditions must be satisfied.

- The player has 300 HP and an attack power of 60. The monster has 100 HP and an attack power of 20.
- Implement attack failure so that the player's attack fails with a 20% probability and the monster's attack fails with a 10% probability.
- Attack order : The player attacks each monster → Each monster attacks the player → ...

When submitting, make sure that your files can be compiled and run using following commands.

Commands

```
$ javac Assignment9B_[Student ID].java GameCharacter.java BattleProcess.java
$ java Assignment9B_[Student ID]
```

提出ファイル名 Submission file name: **Assignment9B_[Student ID].java, GameCharacter.java BattleProcess.java**

提出方法 Submission method: **Moodle**

提出期限 Submission deadline: **第10回講義の開始まで. By the start of the class 10.**

講義内課題 9B In class Assignment 9B

MonsterBattleGame_WithClass_2.java

```
import java.util.Random; // Randomクラスの使用 / Use the Random class
import java.util.Scanner; // Scannerクラスの使用 / Use the Scanner class

public class MonsterBattleGame_WithClass_2 {

    public static void main(String args[]){ // プログラムの実行開始地点 / The starting point of program execution

        // プログラム実行の準備 / Program execution setup
        Scanner scan = new Scanner(System.in); // Enter入力を受け取るための準備 / Prepare for reading Enter input
        Random rand = new Random(999); // 乱数生成処理の初期化 / Initialize random number generation

        // プレイヤー配列の生成 / Create the player array
        GameCharacter[] players = new GameCharacter[1];

        // モンスター配列の生成 / Create the monster array
        GameCharacter[] monsters = new GameCharacter[3];

        // インスタンスの生成と参照 / Creating and referring to instances
        players[0] = new GameCharacter("Player0", 300, 50, 20); // プレイヤー0を生成 / Create the player0
        monsters[0] = new GameCharacter("Monster0", 100, 20, 10); // モンスター0を生成 / Create Monster0
        monsters[1] = new GameCharacter("Monster1", 100, 20, 10); // モンスター1を生成 / Create Monster1
        monsters[2] = new GameCharacter("Monster2", 100, 20, 10); // モンスター2を生成 / Create Monster2

        // ゲーム情報の表示 / Display the game information
        System.out.println("~~~~~");
        System.out.println("    MonsterBattleGame_WithClass_2    ");
        System.out.println("~~~~~");
        for (int i = 0; i < players.length; i++) { // 全プレイヤーについて繰り返す / Repeat for all players
            players[i].print(); // プレイヤー情報の表示 / Display player information
        }
        for (int i = 0; i < monsters.length; i++) { // 全モンスターについて繰り返す / Repeat for all monsters
            monsters[i].print(); // モンスター情報の表示 / Display monster information
        }

        // バトル進行 / Battle control
        System.out.println("# バトル開始 / Battle start");
        while (true) {

            System.out.println("# Enterを入力して攻撃 / Press Enter to attack");
            scan.nextLine(); // プレイヤーの入力待ち / Wait for player input

            // 各プレイヤーが各モンスターを攻撃 / Each player attacks each monster
            BattleProcess.performAttacks(players, monsters, rand);

            // モンスターの戦闘不能判定 / Check whether monsters are defeated
            if (BattleProcess.allDefeated(monsters)) { // 全モンスターが戦闘不能の場合 / If all monsters are defeated
                BattleProcess.printWinner(players[0]); // プレイヤーの勝利 / Players win
                break; // ループを抜ける / Exit the loop
            }

            // 各モンスターが各プレイヤーを攻撃 / Each monster attacks each player
            BattleProcess.performAttacks(monsters, players, rand);

            // プレイヤーの戦闘不能判定 / Check whether players are defeated
            if (BattleProcess.allDefeated(players)) { // 全プレイヤーが戦闘不能の場合 / If all players are defeated
                BattleProcess.printWinner(monsters[0]); // モンスターの勝利 / Monsters win
                break; // ループを抜ける / Exit the loop
            }
        }

        // 終了処理 / Exit handling
        scan.close(); // Scannerを閉じる / Close the Scanner
    }
}
```

講義内課題 9B In class Assignment 9B

GameCharacter.java

```
// =====  
//  
// ゲームキャラクタークラス / Game character class  
//  
// =====  
public class GameCharacter {  
  
    // =====  
    // フィールド / Fields  
    // - インスタンスが持つデータ / Fields are data stored in each instance  
    String name;          // キャラクター名 / Character name  
    int hp;                // キャラクターのHP / Character HP  
    int attack_power;      // キャラクターの攻撃力 / Character Attack power  
    int attack_fails_rate; // キャラクターの攻撃失敗確率 / Character Attack fails probability  
  
    // =====  
    // コンストラクタ / Constructor  
    // - コンストラクタの名前はクラス名と同じにする / The constructor name must be the same as the class name  
    // - コンストラクタはインスタンスを生成するときに最初に実行される / A constructor runs when an instance is created  
    public GameCharacter(String character_name, int character_hp, int character_attack_power, int character_attack_fails_rate) {  
        name = character_name;          // 引数character_nameをフィールドnameに代入 / Assign the argument character_name to the field name  
        hp = character_hp;              // 引数character_hpをフィールドhpに代入 / Assign the argument character_hp to the field hp  
        attack_power = character_attack_power; // 引数character_attack_powerをフィールドattack_powerに代入 / Assign the argument character_attack_power to the field attack_power  
        attack_fails_rate = character_attack_fails_rate; // 引数character_attack_failsをフィールドattack_failsに代入 / Assign the argument character_attack_fails to the field attack_fails  
    }  
  
    // =====  
    // パラメータを表示するメソッド / Method for printing parameters  
    public void print() {  
        System.out.println("[ " + name + " ]");  
        System.out.println("- HP: " + hp);  
        System.out.println("- Attack Power: " + attack_power);  
        System.out.println("- Attack Fails Probability: " + attack_fails_rate);  
        System.out.println("-----");  
    }  
  
    // =====  
    // ダメージを受けるメソッド / Method for receiving damage  
    public void receiveDamage(int damage) {  
        hp -= damage; // 受けたダメージ分だけHPを減らす / Reduce HP by the damage value  
        if (hp < 0) { hp = 0; }  
    }  
}
```

講義内課題 9B In class Assignment 9B

BattleProcess.java (1)

```
import java.util.Random;    // Randomクラスの使用 / Use the Random class

// =====
//
// 戦闘プロセスクラス / Battle process class
//
// - BattleProcessはHPなどのフィールドを持たないため、インスタンスを作らずに使う / Since BattleProcess has no fields such as HP, these methods are used without creating an instance
// - staticを付けると、BattleProcessインスタンスを作らずにメソッドを呼び出せる / By using static, this method can be called without creating a BattleProcess instance
//
// =====
public class BattleProcess {

    // =====
    // 戦闘結果を表示するメソッド / Method for displaying the battle result
    public static void printWinner(GameCharacter winner) {
        System.out.println("");
        System.out.println("-----");
        System.out.println(" 結果 / Battle Result ");
        System.out.println("-----");
        System.out.println("> Team " + winner.name + " wins!");
        System.out.println("-----");
    }

    // =====
    // 攻撃処理メソッド / Attack process method
    // - attackerがtargetを攻撃 / The attacker attacks the target
    public static void attack(GameCharacter attacker, GameCharacter target, Random rand) {

        // 変数定義 / Variable definition
        int percentage_100 = 100;    // 確率判定用乱数の上限 / Upper limit of the random number for probability checks

        // 攻撃者を表示 / Show the attacker
        System.out.println(attacker.name + "の攻撃 / Attack by " + attacker.name);

        // 攻撃失敗 / Attack failure
        if (rand.nextInt(percentage_100) < attacker.attack_fails_rate) {    // 攻撃失敗判定 / Attack failure check
            System.out.println("> 攻撃は失敗した...! / The attack failed...!");
        }

        // 攻撃成功 / Attack success
        } else {

            // 攻撃ダメージの計算 / Calculate attack damage
            int attack_damage = rand.nextInt(attacker.attack_power);

            // 攻撃対象にダメージを与える / Deal damage to the target
            target.receiveDamage(attack_damage);

            // 攻撃結果を表示 / Show attack result
            System.out.println("> " + target.name + "に" + attack_damage + "のダメージ! / Dealt " + attack_damage + " damage to the " + target.name + "!");
            System.out.println("> " + target.name + "の残りHP: " + target.hp + " / " + target.name + " HP: " + target.hp);
        }
    }
}
```

講義内課題 9B In class Assignment 9B

BattleProcess.java (2)

```
// =====
// 攻撃側チームが防御側チームを攻撃するメソッド / Method for performing attacks from attackers to targets
public static void performAttacks(GameCharacter[] attackers, GameCharacter[] targets, Random rand) {

    for (int i = 0; i < attackers.length; i++) {          // 全攻撃側キャラクターについて繰り返す / Repeat for all attackers
        if (!isDefeated(attackers[i])) {                  // 攻撃側が戦闘不能でない場合 / If the attacker is not defeated

            for (int j = 0; j < targets.length; j++) {      // 全防御側キャラクターについて繰り返す / Repeat for all targets
                if (!isDefeated(targets[j])) {              // 防御側が戦闘不能でない場合 / If the target is not defeated
                    attack(attackers[i], targets[j], rand); // 攻撃処理 / Attack process
                }
            }
        }
    }
}

// =====
// 戦闘不能を判定するメソッド / Method for checking whether the character is defeated
public static boolean isDefeated(GameCharacter character) {
    return character.hp <= 0; // HPが0以下ならtrueを返す / Return true if HP is 0 or less
}

// =====
// 全キャラクターが戦闘不能か判定するメソッド / Method for checking whether all characters are defeated
public static boolean allDefeated(GameCharacter[] characters) {

    boolean all_defeated = true; // 全キャラクターが戦闘不能かどうかを表す変数 / Variable that shows whether all characters are defeated

    for (int i = 0; i < characters.length; i++) {          // 全キャラクターについて判定を繰り返す / Repeat defeat check for all characters
        all_defeated = all_defeated && isDefeated(characters[i]);
    }
    return all_defeated; // 判定結果を返す / Return the result
}
}
```

講義内課題 9C In class Assignment 9C

講義内課題Bで工夫した点をパワーポイント形式(スライド5枚以内)で説明しなさい。

Explain the improvements you made in In class Assignment B in PowerPoint format (within 5 slides).

提出ファイル名 Submission file name: Assignment9C_[Student ID].pptx or other PowerPoint filetype

提出方法 Submission method: Moodle

提出期限 Submission deadline: 第10回講義の開始まで. By the start of the class 10.

クラス Class

- データと処理をまとめ、インスタンスを生成するための雛形
A template for grouping data and operations and creating instances.

クラス Class



フィールド Field

名前：早稲田太郎 Name: Taro Waseda

誕生年：1999 BirthYear: 1999

誕生月：5 BirthMonth: 5

誕生日：20 BirthDay: 20

身長：165.8 cm Height: 165.8 cm

メソッド Method

- Walk()
- Run()
- talk()
- ⋮

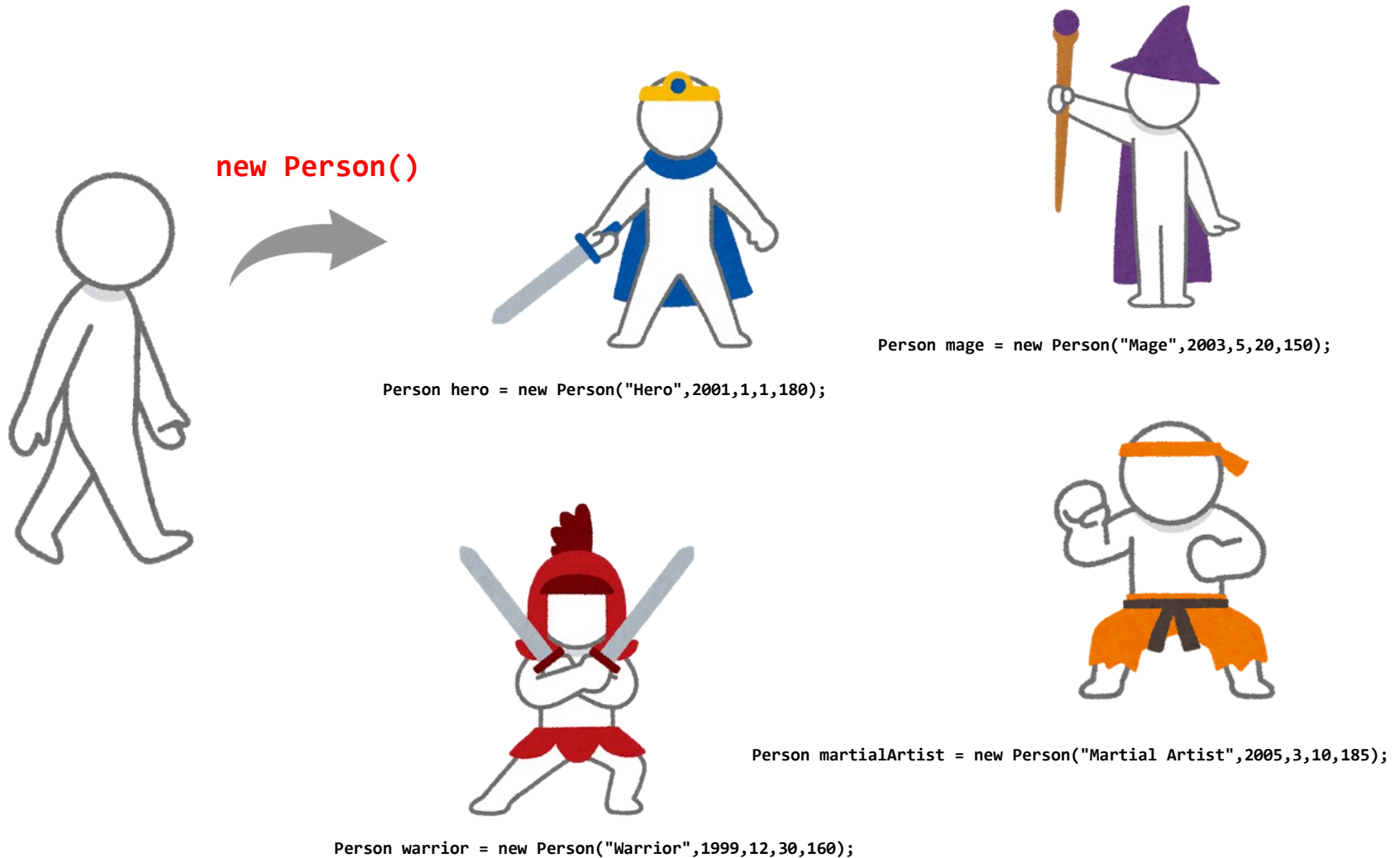
関連のあるデータを
まとめて扱いたい！
Related data should be
handled together.

Class (クラス) : public class Person

```
public class Person {  
  
    // -----  
    // フィールド / Fields  
    // -----  
    String name;  
    int birth_year;  
    int birth_month;  
    int birth_day;  
    double height;  
  
    // -----  
    // コンストラクタ / Constructor (初期化 / Initialization)  
    // -----  
    public Person(String n, int y, int m, int d, double h) {  
        name = n;  
        birth_year = y;  
        birth_month = m;  
        birth_day = d;  
        height = h;  
    }  
  
    // -----  
    // メソッド / Method  
    // -----  
    public void walk() {  
        /* Walking process */  
    }  
  
    public void run() {  
        /* Running process */  
    }  
  
    public void talk() {  
        /* Talking process */  
    }  
}
```

インスタンス Instance

- インスタンスとは, クラスの実体
An instance is an object created from a class.



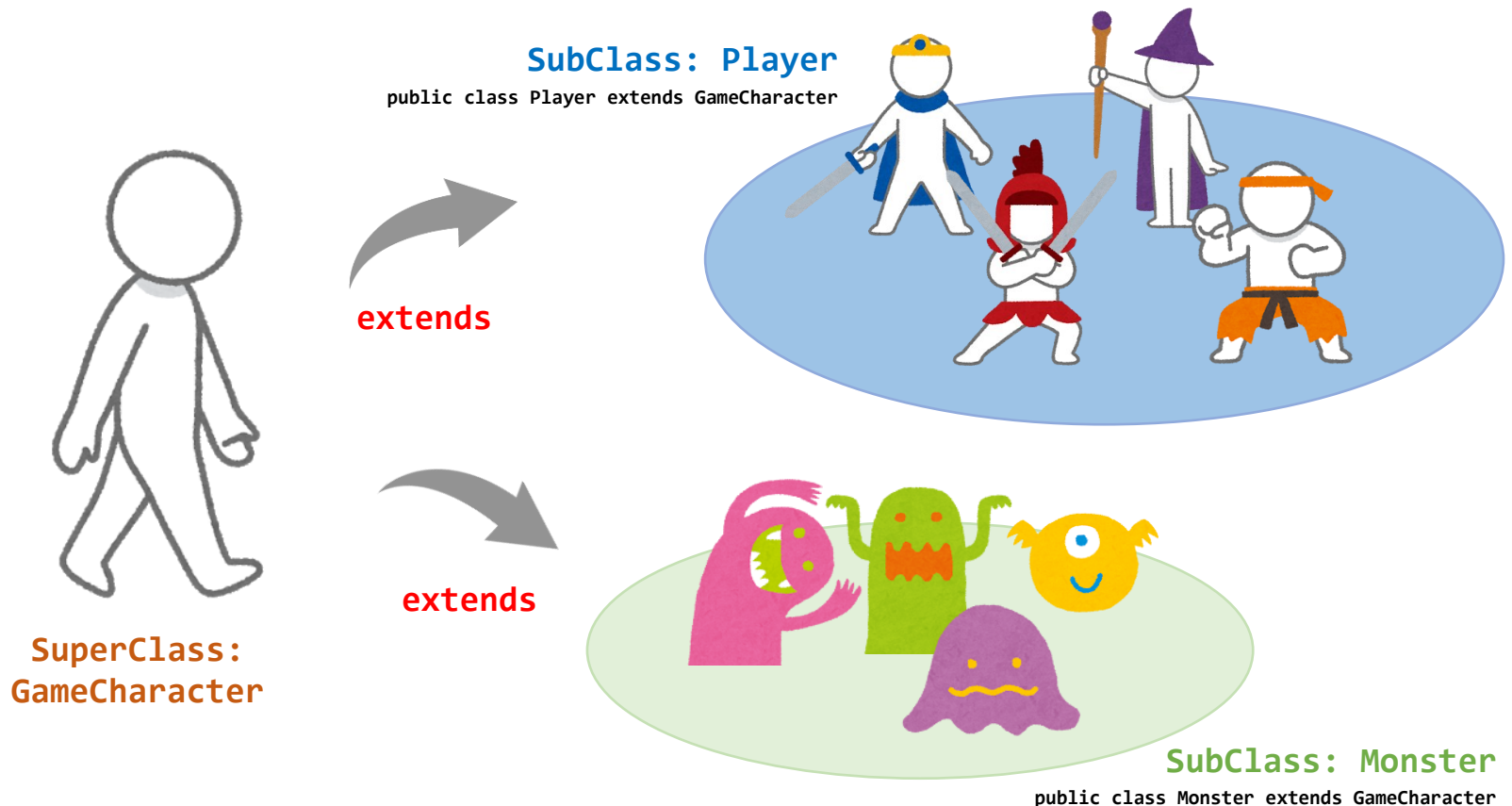


継承・オーバーライド

Inheritance and Overriding

継承 Inheritance

- サブクラスが、スーパークラスのフィールド・メソッドを引き継ぐ
Subclass inherits the fields and methods from a superclass.
- コンストラクタはサブクラスに継承されない
Constructors are not inherited to subclass.



スーパークラス Superclass

- スーパークラスはクラスがextendされたもので、単なるクラスである。
Superclass is an ordinary class.
When class is extended, the class call as Superclass.
- したがって、特別に宣言をする必要はない。
Therefore, no special declaration is required in the superclass.

スーパークラス Superclass



フィールド Field

名前：早稲田太郎 Name: Taro Waseda

誕生年：1999 BirthYear: 1999

メソッド Method

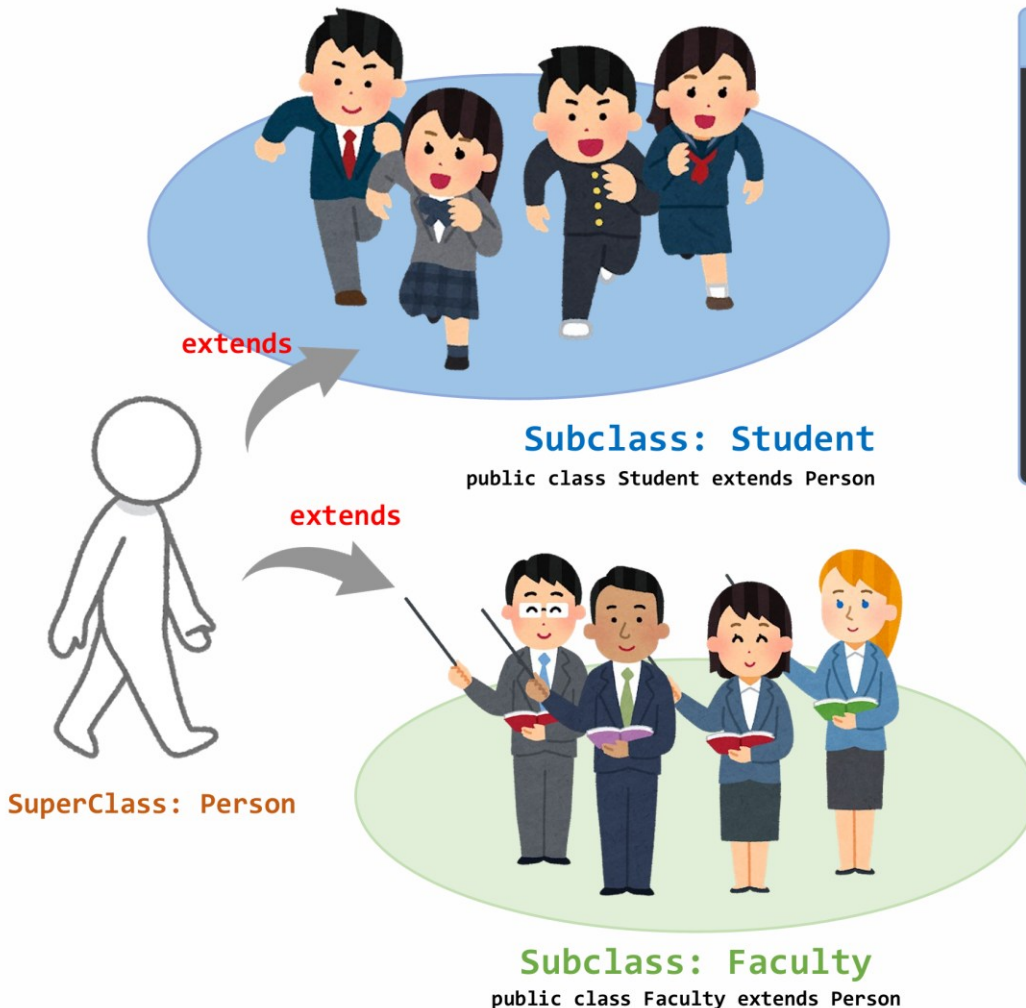
Talk(), Walk(), ...

Person.java (Superclass, スーパークラス)

```
public class Person {  
    // -----  
    // フィールド / Fields  
    // -----  
    String name;  
    int birth_year;  
  
    // -----  
    // コンストラクタ / Constructor (初期化 / Initialization)  
    // -----  
    public Person(String n, int y) {  
        name = n;  
        birth_year = y;  
    }  
  
    // -----  
    // メソッド / Method  
    // -----  
    public void talk() {  
        System.out.println("I'm" + name + ".");  
    }  
    public void walk() {  
        /* Walking process */  
    }  
}
```

サブクラス Subclass

- サブクラスは, extendsによって他のクラスを継承したクラスである。
A subclass is a class that inherits another class using extends.



Student.java (Subclass, サブクラス)

```
public class Student extends Person {  
    // フィールド / Fields  
    int id;  
  
    // コンストラクタ / Constructor  
    public Student(String n, int y, int sid) {  
        super(n, y);  
        id = sid;  
    }  
  
    // メソッド / Method  
    @Override  
    public void talk() {  
        super.talk();  
        System.out.println("My student ID is " + id + ".");  
    }  
}
```

Faculty.java (Subclass, サブクラス)

```
public class Faculty extends Person {  
    // フィールド / Fields  
    int course;  
  
    // コンストラクタ / Constructor  
    public Faculty(String n, int y, String c) {  
        super(n, y);  
        course = c;  
    }  
  
    // メソッド / Method  
    @Override  
    public void talk() {  
        super.talk();  
        System.out.println("My teaching class is " + course + ".");  
    }  
}
```

継承の練習 Practice Using Inheritance



extends

Subclass: Student

public class Student extends Person



extends



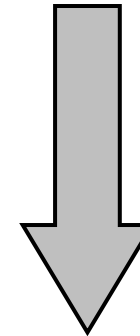
Subclass: Faculty

public class Faculty extends Person

SuperClass: Person

SelfIntroduction.java

```
public class SelfIntroduction {  
    public static void main(String[] args) {  
        Student st1 = new Student("Alice", 2005, 123456);  
        Student st2 = new Student("Bob", 2006, 123457);  
  
        Faculty fc1 = new Faculty("Carol", 1980, "Java Programming");  
        Faculty fc2 = new Faculty("Dave", 1980, "Electromagnetics");  
  
        st1.talk();  
        st2.talk();  
  
        fc1.talk();  
        fc2.talk();  
    }  
}
```



**コンパイル&実行
Compiling & Execution**

課題が終われば退出可能です。
TAさんに自身の座席番号を伝えてからご退出下さい。

You can leave once you have completed the assignment. Please inform
a teaching assistant (TA) of your seat number before leaving.

講義内課題

In class Assignment

Please do not include your CD (check digit)
in the student ID part of the filename.

Example:

Filename Format: Assignment9A_[Student ID].java

Student ID: 1w123456-7

Filename: Assignment9A_1w123456.java

講義内課題 10A In class Assignment 10A

1. “継承の練習”で実行した, **コンパイル**に必要な**コマンド**を回答しなさい.

Answer the **commands** required for **compilation** that were executed in the “Practice Using Inheritance” section.

2. “継承の練習”で実行した, **実行**に必要な**コマンド**を回答しなさい.

Answer the **commands** required for **execution** that were executed in the “Practice Using Inheritance” section.

3. “継承の練習”で得られた, **実行結果**を回答しなさい.

Answer the **execution results** obtained from the “Practice Using Inheritance”.

提出方法 Submission method: **Moodle**

提出期限 Submission deadline: **第11回講義の開始まで. By the start of the class 11.**

講義内課題 10B In class Assignment 10B

- “継承の練習”で作成したファイルを修正して、あなたの 氏名 及び 学籍番号 が正しく表示されるように修正しなさい。

Modify the files created in the “Practice Using Inheritance” section so that your name and student ID are displayed correctly.

- 修正したファイルのみを、ファイル名を変更せずにそのままアップロードして下さい。
Upload **ONLY** the modified files **without changing** their file names.

提出方法 Submission method: **Moodle**

提出期限 Submission deadline: **第11回講義の開始まで. By the start of the class 11.**

講義内課題 10C In class Assignment 10C

1. 講義内課題 9B の回答例として示した下記3ファイルを作成しなさい。作成するプログラムは、回答例と同じ内容・動作になるようにして下さい。日本語/英語の両方で重複して記載されている コメント/print()/println() は、どちらか一方の言語のみで作成しても問題ありません。

Create the following three files shown as the sample answer for In-class Assignment 9B. The created program must have the same content and behavior as the sample answer. For comments/print()/println() written redundantly in both Japanese and English, you may write only one of the two languages.

- 1. MonsterBattleGame_WithClass_2.java
- 2. GameCharacter.java
- 3. BattleProcess.java

2. GameCharacterクラスを継承するサブクラスとしてPlayerクラスとMonsterクラスを作成しなさい。作成したサブクラスを用いて、MonsterBattleGame_WithClass_2.javaが正しく動作するようにプログラムを修正しなさい。作成・修正した最終版のファイルをZipファイルに圧縮したうえでMoodleに提出して下さい。また修正した箇所と内容を説明しなさい。

Create the Player class and the Monster class as subclasses that extend the GameCharacter class. Modify MonsterBattleGame_WithClass_2.java so that the program works correctly using the created subclasses.

Compress the final versions of the created and modified files into a Zip file, and submit the Zip file on Moodle. Also explain which parts you modified and how you modified them.

提出ファイル名 Submission file name : Assignment10C_[Student ID].zip

提出方法 Submission method : Moodle

提出期限 Submission deadline : 第11回講義の開始まで。 By the start of the class 11.